

Phase_1_Extraction_Slovenia_SII_demo

November 4, 2025

1 Slovenia Solvency reports Table S.02.01.02; Part 1 Extraction

The scope of this script is to transcribe SFCR table S.02.01.02 for the 7 life insurance companies on the Slovenian market. The notebook is organized into the following sections:

- 1) Companies and tables in scope
- 2) Packages and tools
- 3) Generation of DataFrames

1.1 Companies in scope

For the year 2024, the companies in scope are the following:

- Zavarovalnica Triglav d.d.
- Generali zavarovalnica d.d.
- Prva osebna zavarovalnica, d.d.
- Modra zavarovalnica d.d.
- Vzajemna d.d.
- Grawe zavarovalnica d.d.
- Zavarovalnica Sava, d.d.

1.1.1 Solvency and Financial condition reports

According to Article 51 of the Solvency II Directive 2009/138/EC, companies under the regulatory umbrella of EIOPA, companies must publish annually a Solvency and Financial Condition Reports (SFCR) for all legal entities.

Part of the report is mandatory tables that show some financial and actuarial indicators. One such table is S.02.01.02 which shows a simplified balance sheet of the legal entity. This table is inside the scope of this demo.

1.2 Description of the process

The process of extraction is performed in 5 phases:

1.2.1 Phase 1: Find the reports and identify the relevant tables.

- 1) Identify the new SFCR report and save it into the folder Input.
- 2) Identify the pages where the tables of interest are.
- 3) Compile the map of the company run in the master_list.csv.

1.2.2 Phase 2: Run the Extraction script (this script).

The script performs the following steps (with slight modifications depending on the table format):
1) Save the page with the table into a separate folder Single_pdf. 2) Use either a Python package or specialized LLM to create a digital equivalent of the table. 3) Fix the systemic errors that prevent the table from being saved as DataFrame. 4) Save the DataFrame into the Output folder.

1.2.3 Phase 3: Run the Processing script.

The script applies fixes to the DataFrame to make the numbers closer to the reported numbers. It joins all the tables into a single dataset.

1.2.4 Phase 4: Run the Cross-Validation script.

Applies a series of tests that check for the internal consistency between the numbers. Flags the potential errors.

1.2.5 Phase 5: Final modifications to the table and a manual inspection.

1.3 Necessary Python packages

```
[2]: pip install PyPDF2 pdfplumber
```

```
Requirement already satisfied: PyPDF2 in c:\users\grego\anaconda3\lib\site-packages (3.0.1)
Requirement already satisfied: pdfplumber in c:\users\grego\anaconda3\lib\site-packages (0.11.7)
Requirement already satisfied: pdfminer.six==20250506 in c:\users\grego\anaconda3\lib\site-packages (from pdfplumber) (20250506)
Requirement already satisfied: Pillow>=9.1 in c:\users\grego\anaconda3\lib\site-packages (from pdfplumber) (10.3.0)
Requirement already satisfied: pypdfium2>=4.18.0 in c:\users\grego\anaconda3\lib\site-packages (from pdfplumber) (4.30.0)
Requirement already satisfied: charset-normalizer>=2.0.0 in c:\users\grego\anaconda3\lib\site-packages (from pdfminer.six==20250506->pdfplumber) (2.0.4)
Requirement already satisfied: cryptography>=36.0.0 in c:\users\grego\anaconda3\lib\site-packages (from pdfminer.six==20250506->pdfplumber) (42.0.5)
Requirement already satisfied: cffi>=1.12 in c:\users\grego\anaconda3\lib\site-packages (from cryptography>=36.0.0->pdfminer.six==20250506->pdfplumber) (1.16.0)
Requirement already satisfied: pycparser in c:\users\grego\anaconda3\lib\site-packages (from cffi>=1.12->cryptography>=36.0.0->pdfminer.six==20250506->pdfplumber) (2.21)
Note: you may need to restart the kernel to use updated packages.
```

```
[3]: !pip install mistralai
```

Requirement already satisfied: mistralai in c:\users\grego\anaconda3\lib\site-packages (1.9.3)

Requirement already satisfied: eval-type-backport>=0.2.0 in c:\users\grego\anaconda3\lib\site-packages (from mistralai) (0.2.2)

Requirement already satisfied: httpx>=0.28.1 in c:\users\grego\anaconda3\lib\site-packages (from mistralai) (0.28.1)

Requirement already satisfied: pydantic>=2.10.3 in c:\users\grego\anaconda3\lib\site-packages (from mistralai) (2.11.7)

Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\grego\anaconda3\lib\site-packages (from mistralai) (2.9.0.post0)

Requirement already satisfied: typing-inspection>=0.4.0 in c:\users\grego\anaconda3\lib\site-packages (from mistralai) (0.4.1)

Requirement already satisfied: anyio in c:\users\grego\anaconda3\lib\site-packages (from httpx>=0.28.1->mistralai) (4.2.0)

Requirement already satisfied: certifi in c:\users\grego\anaconda3\lib\site-packages (from httpx>=0.28.1->mistralai) (2025.8.3)

Requirement already satisfied: httpcore==1.* in c:\users\grego\anaconda3\lib\site-packages (from httpx>=0.28.1->mistralai) (1.0.9)

Requirement already satisfied: idna in c:\users\grego\anaconda3\lib\site-packages (from httpx>=0.28.1->mistralai) (3.7)

Requirement already satisfied: h11>=0.16 in c:\users\grego\anaconda3\lib\site-packages (from httpcore==1.*->httpx>=0.28.1->mistralai) (0.16.0)

Requirement already satisfied: annotated-types>=0.6.0 in c:\users\grego\anaconda3\lib\site-packages (from pydantic>=2.10.3->mistralai) (0.6.0)

Requirement already satisfied: pydantic-core==2.33.2 in c:\users\grego\anaconda3\lib\site-packages (from pydantic>=2.10.3->mistralai) (2.33.2)

Requirement already satisfied: typing-extensions>=4.12.2 in c:\users\grego\anaconda3\lib\site-packages (from pydantic>=2.10.3->mistralai) (4.14.1)

Requirement already satisfied: six>=1.5 in c:\users\grego\anaconda3\lib\site-packages (from python-dateutil>=2.8.2->mistralai) (1.16.0)

Requirement already satisfied: sniffio>=1.1 in c:\users\grego\anaconda3\lib\site-packages (from anyio->httpx>=0.28.1->mistralai) (1.3.0)

[4]: !pip install pycryptodome

Requirement already satisfied: pycryptodome in c:\users\grego\anaconda3\lib\site-packages (3.23.0)

1.4 Python packages

```
[5]: import re
import os
import json
import base64
import pdfplumber
import pandas as pd
import numpy as np
from pathlib import Path
from PyPDF2 import PdfReader, PdfWriter
from mistralai.models import OCRResponse
from IPython.display import Markdown, display
from mistralai import Mistral, DocumentURLChunk, ImageURLChunk, TextChunk
```

1.5 API key

The model of choice for this project is Mistral. This was identified as the most economical choice. Additionally, Mistral released a custom model for OCR tasks.

Useful links:

- <https://docs.mistral.ai/getting-started/quickstart/>
- <https://mistral.ai/news/mistral-ocr>

```
[115]: #api_key = "[YOUR MISTRAL API KEY]"
```

1.6 Functions

```
[7]: def set_code_index_and_save(table: pd.DataFrame, path: str) -> None:
    """
    Set the 'CODE' column as the index of a DataFrame and save it as a CSV file.
    """
    table = table.set_index("CODE")
    table.to_csv(path)
```

```
[8]: def extract_tables_from_pdf(pdf_path: str, page_number:int=1) -> pd.DataFrame:
    """
    Extract the first table from a specific page of a PDF and return it as a
    DataFrame.

    Parameters
    -----
    pdf_path : str or pathlib.Path
        Path to the PDF file.
    page_number : int, optional (default=1)
        The page number to extract the table from (1-indexed, i.e., first page
        is 1).
```

```

Returns
-----
pd.DataFrame
    A DataFrame containing the extracted table. The first row of the table
    is
    treated as the header, and the remaining rows as data.

"""

with pdfplumber.open(pdf_path) as pdf:
    page = pdf.pages[page_number - 1] # Pages are zero-indexed
    tables = page.extract_tables()
    df_tables = pd.DataFrame(tables[0][1:], columns=tables[0][0])
    return df_tables

```

```

[9]: def extract_page(input_pdf_path: str, output_pdf_path: str, page_number: int,
    password: str = "") -> None:
    """
    Extract a single page from a PDF file and save it as a new PDF.
    """

    pdf_reader = PdfReader(input_pdf_path)
    pdf_writer = PdfWriter()

    # Decrypt if necessary
    if pdf_reader.is_encrypted:
        if password:
            pdf_reader.decrypt(password)
        else:
            pdf_reader.decrypt("") # try empty password

    # Add the specified page to the PdfWriter object
    pdf_writer.add_page(pdf_reader.pages[page_number - 1])

    # Write the selected page to a new PDF file
    with open(output_pdf_path, 'wb') as output_pdf_file:
        pdf_writer.write(output_pdf_file)

```

```

[10]: def encode_pdf(pdf_path: str) -> None:
    """Encode the pdf to base64."""
    try:
        with open(pdf_path, "rb") as pdf_file:
            return base64.b64encode(pdf_file.read()).decode('utf-8')
    except FileNotFoundError:
        print(f"Error: The file {pdf_path} was not found.")
        return None

```

```

except Exception as e: # Added general exception handling
    print(f"Error: {e}")
    return None

```

```

[11]: def markdown_table_to_dataframe(markdown_text, shift_row = 0):
    """
    Convert a Markdown-formatted table into a pandas DataFrame.
    """

    # Split the markdown text into lines
    lines = markdown_text.strip().split('\n')

    # Filter out lines that are part of the table
    table_lines = [line for line in lines if line.startswith('|')]

    # Remove the markdown table syntax
    cleaned_lines = [line.strip('|').strip() for line in table_lines]

    # Split each line into columns
    data = [line.split('|') for line in cleaned_lines]

    # Extract headers and rows
    headers = [header.strip() for header in data[1+shift_row] if header.strip()]
    rows = [[cell.strip() for cell in row if cell.strip()] for row in data[3:]]

    # Ensure each row has the same number of columns as the headers
    for row in rows:
        if len(row) < len(headers):
            row += [''] * (len(headers) - len(row))

    # Create a DataFrame
    df = pd.DataFrame(rows, columns=headers)
    return df

```

```

[12]: def split_strings_to_df(strings):
    """
    Convert a list of strings in the format `text ..... number` into a
    ↪ DataFrame.

    Each string is expected to contain a text label followed by dots and a
    ↪ numeric value.

    The numeric value is cleaned (spaces and dots removed) and converted to
    ↪ either
    `int` or `float`. If no numeric value is found, `None` is used instead.

    Parameters
    -----

```

strings : list of str
 A list of strings to be parsed.
 Example: ["Technical provisions 2.337.991", "Best Estimate
 ↪0"]

Returns

pd.DataFrame

A DataFrame with two columns:

- "NAME": str, the extracted text (trimmed of dots and whitespace).
- "CO010": int, float, or None, the extracted numeric value.

Notes

- Numbers with thousand separators like `2.337.991` are cleaned into
 ↪integers (`2337991`).
- If the number cannot be parsed, it remains as a string.
- Strings without a numeric value will have `None` in the `CO010` column.

```
data = []
for s in strings:
    # Try to match "text ..... number"
    match = re.match(r'^(.*?)\.{2,}\s*([\d\s\.]+)$', s.strip())
    if match:
        left = match.group(1).strip()
        # Clean up number (remove spaces in middle, convert to float/int)
        right = match.group(2).replace(" ", "").replace(".", "")
        if right.isdigit():
            right = int(right)
        else:
            try:
                right = float(right)
            except:
                pass
        data.append((left, right))
    else:
        # No number found, just keep text
        data.append((s.strip(), None))

df = pd.DataFrame(data, columns=["NAME", "CO010"])
return df
```

```
[13]: def run_mistral_ocr(output_pdf_path: str, api_key: str)-> str:
        """
        Run OCR on a PDF file using the Mistral OCR API and return the extracted
        ↪text in Markdown format.
```

```

The function:
1. Encodes the PDF as a base64 string.
2. Sends it to the Mistral OCR API.
3. Extracts the recognized text in Markdown format from the response.
"""

# Getting the base64 string
base64_pdf = encode_pdf(Path(output_pdf_path))

client = Mistral(api_key=api_key)

ocr_response = client.ocr.process(
    model="mistral-ocr-latest",
    document={
        "type": "document_url",
        "document_url": f"data:application/pdf;base64,{base64_pdf}"
    },
    include_image_base64=True
)

# Assuming ocr_response is your instance of OCRResponse
markdown_texts = [page.markdown for page in ocr_response.pages]

# If you want to concatenate all markdown texts from all pages
full_markdown_text = "\n".join(markdown_texts)

# Assuming ocr_response is your OCRResponse object
markdown_text = ocr_response.pages[0].markdown
return markdown_text

```

```

[14]: def parse_table_to_df(text: str, value_column_name:str) -> pd.DataFrame:
    """
    Parse a long text table into a DataFrame with columns:
    ['code', 'name', 'value'].
    """
    # Split text into lines
    lines = text.splitlines()

    rows = []
    buffer = []
    for line in lines:
        # Match lines with pattern: code (Rxxxx) followed by number
        match = re.match(r"^(R\d{4})\s*(.*)$", line)
        if match:
            code = match.group(1)

```



```

rest = match.group(2).strip()

# Check if 'rest' is a number (the value)
if re.match(r"^\d\.$", rest):
    name = " ".join(buffer).strip()
    value = rest
    rows.append((code, name, value))
    buffer = [] # reset for next name
else:
    # It's not just a number, so treat it as name continuation
    buffer.append(line)
else:
    # If line is just a number after a code line
    if re.match(r"^\d\.$", line.strip()):
        name = " ".join(buffer).strip()
        value = line.strip()
        # Last buffer entry should contain the code
        code_match = re.search(r"(R\d{4})", buffer[-1]) if buffer else None
        code = code_match.group(1) if code_match else None
        # Remove code from name
        buffer[-1] = buffer[-1].replace(code, "").strip() if code else buffer[-1]
        rows.append((code, " ".join(buffer).strip(), value))
        buffer = [] # reset
    else:
        # Accumulate description
        buffer.append(line)

# Build DataFrame
df = pd.DataFrame(rows, columns=["CODE", "NAME", value_column_name])

# Convert numeric values to float
df[value_column_name] = df[value_column_name].str.replace(".", "", regex=False).str.replace(",", ".", regex=False)
df[value_column_name] = pd.to_numeric(df[value_column_name], errors="coerce")

return df

```

```

[15]: def extract_paths(master_list: pd.DataFrame, unique_id: str):
    """
    Extract file paths and page number from master_list for a given unique_id.

    Parameters:
        master_list (pd.DataFrame): DataFrame containing metadata.
        unique_id: Index or identifier in the DataFrame.
    """

```

```

Returns:
    dict: Dictionary with keys 'pdf_path', 'page_number',
          'output_pdf_path', 'output_final_path', 'codes_path'.
    """
    return master_list.loc[unique_id, "document_name"], int(master_list.
↪loc[unique_id, "page_number"]), master_list.loc[unique_id,
↪"output_pdf_path"], master_list.loc[unique_id, "output_final_path"],
↪master_list.loc[unique_id, "codes_path"]

```

```

[16]: def convert_to_dataframe(text: str) -> pd.DataFrame:
    """
    Convert raw Annex I/Solvency II balance sheet text into a structured
    ↪DataFrame.

    Parameters
    -----
    text : str
        Raw text containing balance sheet items.

    Returns
    -----
    pd.DataFrame
        DataFrame with columns: CODE, NAME, VALUE
    """
    rows = []

    # Split text into lines
    for line in text.splitlines():
        # Match rows with a code like R001 and a value at the end
        match = re.match(r"^(.*)\s+(R\d+)\s+([-]?[d[\d\.\,]*)$", line.strip())
        if match:
            name = match.group(1).strip()
            code = match.group(2).strip()
            value_str = match.group(3).replace(".", "") # remove thousand
            ↪separators
            value = float(value_str.replace(",", ".")) # convert to float
            rows.append((code, name, value))

    df = pd.DataFrame(rows, columns=["CODE", "NAME", "VALUE"])
    return df

```

```

[17]: def append_zero_if_len4(df: pd.DataFrame, col: str = "CODE") -> pd.DataFrame:
    """
    Append '0' to the code if its length is 4 characters.

    Parameters

```

```

-----
df : pd.DataFrame
    Input dataframe containing the code column.
col : str
    Column name for codes (default = 'CODE').

Returns
-----
pd.DataFrame
    DataFrame with modified codes.
"""
df = df.copy()
df[col] = df[col].apply(lambda x: x + "0" if len(x) == 4 else x)
return df

```

```

[18]: def run_ocr_and_convert_to_df(path: str, api_key: str) -> pd.DataFrame:
        """
        Run OCR on a PDF file and convert the extracted Markdown table into a
        ↪ DataFrame.
        """

        markdown_text = run_mistral_ocr(path, api_key)
        table = markdown_table_to_dataframe(markdown_text)
        return table

```

```

[19]: def ocr_to_dataframe(ocr_text: str) -> pd.DataFrame:
        """
        Converts OCR extracted financial text into a structured DataFrame.

        Extracts description, code (Rxxxx, Cxxxx), and numeric value if present.
        """
        rows = []

        # Split text into lines
        for line in ocr_text.splitlines():
            # Look for patterns like "..... R0030 ..... 123.456"
            match = re.match(r"^(.*?)\.{2,}\s*(R\d{4}|C\d{4})(?:\s*\.{2,}\s*([\d\.\s-]+))?$", line.strip())
            ↪ if match:
                desc = match.group(1).strip()
                code = match.group(2).strip()
                raw_value = match.group(3)

                if raw_value:
                    # Remove spaces inside numbers (e.g. "4.275 .590" -> "4.275.
                    ↪ 590")
                    cleaned = raw_value.replace(" ", "")

```

```

        # Convert to float if numeric, else keep as string
        try:
            value = float(cleaned.replace(".", "").replace(",", "."))
        except ValueError:
            value = cleaned
        else:
            value = None

        rows.append((code, desc, value))

df = pd.DataFrame(rows, columns=["CODE", "DESCRIPTION", "VALUE"])
return df

```

```

[20]: def ocr_text_to_dataframe(text: str) -> pd.DataFrame:
    """
    Transforms OCR-extracted text containing '.....' separators into a clean
    DataFrame.

    Parameters
    -----
    text : str
        OCR-extracted text containing lines with labels and numeric values.

    Returns
    -----
    pd.DataFrame
        A DataFrame with columns ['Description', 'Value'].
        - Description: textual part
        - Value: numeric value (float) or NaN if missing
    """

    # Step 1: Clean up spaces and join wrapped lines
    text = re.sub(r'\s*\n\s*', '\n', text.strip()) # normalize line breaks
    text = re.sub(r' +', ' ', text) # collapse multiple spaces
    text = text.replace('\u202f', '').replace('\xa0', '') # remove
    ↪non-breaking spaces

    # Step 2: Split text into lines
    lines = text.split("\n")

    # Step 3: Merge broken lines (those without numeric values)
    merged_lines = []
    buffer = ""
    for line in lines:
        # if line has no number, append it to previous description
        if not re.search(r'\d', line):
            buffer += " " + line.strip()

```

```

    else:
        if buffer:
            line = buffer.strip() + " " + line.strip()
            buffer = ""
        merged_lines.append(line.strip())

# Step 4: Extract description and numeric value
data = []
for line in merged_lines:
    # Match pattern like "text ..... number"
    match = re.match(r"^(.*?)\.{2,}\s*([\d\.\s]+)$", line)
    if match:
        desc, val = match.groups()
    else:
        # Try to separate trailing numbers without dots
        match = re.match(r"^(.*?)([\d\.\s]+)$", line)
        if match:
            desc, val = match.groups()
        else:
            desc, val = line, np.nan

    # Clean description and numeric value
    desc = desc.strip(" .")
    if isinstance(val, str):
        val = re.sub(r"^[^d,\.]", "", val)
        val = val.replace(" ", "").replace(",", ".")
        try:
            val = float(val)
        except ValueError:
            val = np.nan

    data.append((desc, val))

df = pd.DataFrame(data, columns=["Description", "Value"])
return df

```

```

[21]: def parse_table_to_df(text: str, value_column_name:str) -> pd.DataFrame:
    """
    Parse a long text table into a DataFrame with columns:
    ['code', 'name', 'value'].
    """
    # Split text into lines
    lines = text.splitlines()

    rows = []
    buffer = []
    for line in lines:

```

```

# Match lines with pattern: code (Rxxxx) followed by number
match = re.match(r"^(R\d{4})\s*(.*)$", line)
if match:
    code = match.group(1)
    rest = match.group(2).strip()

    # Check if 'rest' is a number (the value)
    if re.match(r"^\d\.$", rest):
        name = " ".join(buffer).strip()
        value = rest
        rows.append((code, name, value))
        buffer = [] # reset for next name
    else:
        # It's not just a number, so treat it as name continuation
        buffer.append(line)
else:
    # If line is just a number after a code line
    if re.match(r"^\d\.$", line.strip()):
        name = " ".join(buffer).strip()
        value = line.strip()
        # Last buffer entry should contain the code
        code_match = re.search(r"(R\d{4})", buffer[-1]) if buffer else None
        code = code_match.group(1) if code_match else None
        # Remove code from name
        buffer[-1] = buffer[-1].replace(code, "").strip() if code else None
        rows.append((code, " ".join(buffer).strip(), value))
        buffer = [] # reset
    else:
        # Accumulate description
        buffer.append(line)

# Build DataFrame
df = pd.DataFrame(rows, columns=["CODE", "DESCRIPTION", value_column_name])

# Convert numeric values to float
df[value_column_name] = df[value_column_name].str.replace(".", "", regex=False).str.replace(",", ".", regex=False)
df[value_column_name] = pd.to_numeric(df[value_column_name], errors="coerce")

return df

```

1.7 The list of companies

```
[22]: master_list = pd.read_csv("master_list.csv", header=0, index_col=0)
```

```
[23]: display(master_list)
```

	company	document_name \
VZAJEMNA		
GENERALI_02_1	GENERALI	Input\Porocilo-o-solventnosti-in-financnem-pol...
GENERALI_02_2	GENERALI	Input\Porocilo-o-solventnosti-in-financnem-pol...
TRIGLAV_02_1	TRIGLAV	Input\Poročilo+o+solventnosti+in+finančnem+pol...
TRIGLAV_02_2	TRIGLAV	Input\Poročilo+o+solventnosti+in+finančnem+pol...
PRVA_02_1	PRVA	Input\Prva-osebna-zavarovalnica-PSFP_2024-2025...
PRVA_02_2	PRVA	Input\Prva-osebna-zavarovalnica-PSFP_2024-2025...
MODRA_02_1	MODRA	Input\Revidirano-SFCR-porocilo-2024.pdf
MODRA_02_2	MODRA	Input\Revidirano-SFCR-porocilo-2024.pdf
MODRA_02_3	MODRA	Input\Revidirano-SFCR-porocilo-2024.pdf
VZAJEMNA_02_1	VZAJEMNA	Input\Vzajemna_2024.pdf
VZAJEMNA_02_2	VZAJEMNA	Input\Vzajemna_2024.pdf
GRAWE_02_1	GRAWE	Input\Grawe_2024.pdf
GRAWE_02_2	GRAWE	Input\Grawe_2024.pdf
SAVA_02_1	SAVA	Input\Sava-za-leto-2024.pdf
SAVA_02_2	SAVA	Input\Sava-za-leto-2024.pdf

	table_name	page_number \
VZAJEMNA		
GENERALI_02_1	S.02.01.02_A	83
GENERALI_02_2	S.02.01.02_L	84
TRIGLAV_02_1	S.02.01.02_A	110
TRIGLAV_02_2	S.02.01.02_L	111
PRVA_02_1	S.02.01.02_A	63
PRVA_02_2	S.02.01.02_L	64
MODRA_02_1	S.02.01.02_A	83
MODRA_02_2	S.02.01.02_L	84
MODRA_02_3	S.02.01.02_L2	85
VZAJEMNA_02_1	S.02.01.02_A	72
VZAJEMNA_02_2	S.02.01.02_L	73
GRAWE_02_1	S.02.01.02_A	112
GRAWE_02_2	S.02.01.02_L	113
SAVA_02_1	S.02.01.02_A	104
SAVA_02_2	S.02.01.02_L	105

	output_pdf_path \
VZAJEMNA	
GENERALI_02_1	Single_pdf/GENERALI_S02_01_02_1_2024.pdf
GENERALI_02_2	Single_pdf/GENERALI_S02_01_02_2_2024.pdf
TRIGLAV_02_1	Single_pdf/TRIGLAV_S02_01_02_1_2024.pdf
TRIGLAV_02_2	Single_pdf/TRIGLAV_S02_01_02_2_2024.pdf

PRVA_02_1	Single_pdf/PRVA_S02_01_02_1_2024.pdf
PRVA_02_2	Single_pdf/PRVA_S02_01_02_2_2024.pdf
MODRA_02_1	Single_pdf/MODRA_S02_01_02_1_2024.pdf
MODRA_02_2	Single_pdf/MODRA_S02_01_02_2_2024.pdf
MODRA_02_3	Single_pdf/MODRA_S02_01_02_3_2024.pdf
VZAJEMNA_02_1	Single_pdf/VZAJEMNA_S02_01_02_1_2024.pdf
VZAJEMNA_02_2	Single_pdf/VZAJEMNA_S02_01_02_2_2024.pdf
GRAWE_02_1	Single_pdf/GRAWE_S02_01_02_1_2024.pdf
GRAWE_02_2	Single_pdf/GRAWE_S02_01_02_2_2024.pdf
SAVA_02_1	Single_pdf/SAVA_S02_01_02_1_2024.pdf
SAVA_02_2	Single_pdf/SAVA_S02_01_02_2_2024.pdf

output_final_path \

VZAJEMNA	
GENERALI_02_1	Output/GENERALI_S02_01_02_1_2024.csv
GENERALI_02_2	Output/GENERALI_S02_01_02_2_2024.csv
TRIGLAV_02_1	Output/TRIGLAV_S02_01_02_1_2024.csv
TRIGLAV_02_2	Output/TRIGLAV_S02_01_02_2_2024.csv
PRVA_02_1	Output/PRVA_S02_01_02_1_2024.csv
PRVA_02_2	Output/PRVA_S02_01_02_2_2024.csv
MODRA_02_1	Output/MODRA_S02_01_02_1_2024.csv
MODRA_02_2	Output/MODRA_S02_01_02_2_2024.csv
MODRA_02_3	Output/MODRA_S02_01_02_3_2024.csv
VZAJEMNA_02_1	Output/VZAJEMNA_S02_01_02_1_2024.csv
VZAJEMNA_02_2	Output/VZAJEMNA_S02_01_02_2_2024.csv
GRAWE_02_1	Output/GRAWE_S02_01_02_1_2024.csv
GRAWE_02_2	Output/GRAWE_S02_01_02_2_2024.csv
SAVA_02_1	Output/SAVA_S02_01_02_1_2024.csv
SAVA_02_2	Output/SAVA_S02_01_02_2_2024.csv

codes_path leto

VZAJEMNA		
GENERALI_02_1	Codes/Codes_S02_A.csv	2024
GENERALI_02_2	Codes/Codes_S02_L.csv	2024
TRIGLAV_02_1	Codes/Codes_S02_A_Triglav.csv	2024
TRIGLAV_02_2	Codes/Codes_S02_L_Triglav.csv	2024
PRVA_02_1	NaN	2024
PRVA_02_2	NaN	2024
MODRA_02_1	NaN	2024
MODRA_02_2	NaN	2024
MODRA_02_3	NaN	2024
VZAJEMNA_02_1	NaN	2024
VZAJEMNA_02_2	NaN	2024
GRAWE_02_1	NaN	2024
GRAWE_02_2	NaN	2024
SAVA_02_1	NaN	2024
SAVA_02_2	NaN	2024

2 Code

2.1 GENERALI

S.02.01.02 1

```
[24]: unique_id = "GENERALI_02_1"
pdf_path, page_number, output_pdf_path, output_final_path, codes_path =   
↳extract_paths(master_list=master_list, unique_id=unique_id)
extract_page(input_pdf_path=pdf_path, output_pdf_path=output_pdf_path,   
↳page_number=page_number, password = "")

[25]: codes_S020102_A = pd.read_csv(codes_path,header = 0, index_col=0)

[26]: table = extract_tables_from_pdf(pdf_path=output_pdf_path, page_number=1)

[27]: table_e = pd.concat([codes_S020102_A,table], ignore_index=True, axis = 1)

[28]: table_e = table_e.iloc[:, [0,1,4]]

[29]: table_e.columns = ["DESCRIPTION","CODE", "C0010"]

[30]: set_code_index_and_save(table=table_e, path=output_final_path)

[31]: del table, table_e
```

S.02.01.02 2

```
[32]: unique_id = "GENERALI_02_2"
pdf_path, page_number, output_pdf_path, output_final_path, codes_path =   
↳extract_paths(master_list=master_list, unique_id=unique_id)
extract_page(input_pdf_path=pdf_path, output_pdf_path=output_pdf_path,   
↳page_number=page_number, password = "")

[33]: codes_S020102_L = pd.read_csv(codes_path,header = 0, index_col=0)

[34]: table = extract_tables_from_pdf(pdf_path=output_pdf_path, page_number=1)

[35]: table = table.drop(index=(38))

[36]: table=table.reset_index(drop=True)

[37]: table_e = pd.concat([codes_S020102_L,table], ignore_index=True, axis = 1)

[38]: table_e = table_e.iloc[:,[0,1,4]]

[39]: table_e.columns = ["DESCRIPTION","CODE", "C0010"]

[40]: set_code_index_and_save(table=table_e, path=output_final_path)

[41]: del table, table_e
```

2.2 Triglav

S.02.01.02 1

```
[42]: unique_id = "TRIGLAV_02_1"
pdf_path, page_number, output_pdf_path, output_final_path, codes_path =   
↳extract_paths(master_list=master_list, unique_id=unique_id)
extract_page(input_pdf_path=pdf_path, output_pdf_path=output_pdf_path,  
↳page_number=page_number, password = "")

[43]: table = run_ocr_and_convert_to_df(path=output_pdf_path, api_key=api_key)

[44]: codes_S020102_A = pd.read_csv(codes_path,header = 0, index_col=0)

[45]: table = table.drop(index=(25))
table=table.reset_index(drop=True)

[46]: table_e = pd.concat([codes_S020102_A,table], ignore_index=True, axis = 1)

[47]: table_e = table_e.iloc[:,[0,1,3]]

[48]: table_e.columns = ["DESCRIPTION","CODE", "C0010"]

[49]: set_code_index_and_save(table=table_e, path=output_final_path)

[50]: del table, table_e
```

S.02.01.02 2

```
[51]: unique_id = "TRIGLAV_02_2"
pdf_path, page_number, output_pdf_path, output_final_path, codes_path =   
↳extract_paths(master_list=master_list, unique_id=unique_id)
extract_page(input_pdf_path=pdf_path, output_pdf_path=output_pdf_path,  
↳page_number=page_number, password = "")

[52]: markdown_text = run_mistral_ocr(output_pdf_path, api_key)

[53]: table = ocr_text_to_dataframe(markdown_text)

[54]: codes_S020102_L = pd.read_csv(codes_path,header = 0, index_col=0)

[55]: table_e = pd.concat([codes_S020102_L,table], ignore_index=True, axis = 1)

[56]: table_e = table_e.iloc[:,[0,1,3]]

[57]: table_e.columns = ["DESCRIPTION","CODE", "C0010"]

[58]: set_code_index_and_save(table=table_e, path=output_final_path)
```

2.3 Prva osebna zavarovalnica, d.d.

S.02.01.02 1

```
[59]: unique_id = "PRVA_02_1"
pdf_path, page_number, output_pdf_path, output_final_path, codes_path =   
↳extract_paths(master_list=master_list, unique_id=unique_id)
extract_page(input_pdf_path=pdf_path, output_pdf_path=output_pdf_path,  
↳page_number=page_number, password = "")

[60]: table = run_ocr_and_convert_to_df(path=output_pdf_path, api_key=api_key)

[61]: table.columns = ["CODE", "DESCRIPTION", "C0010"]

[62]: set_code_index_and_save(table=table, path=output_final_path)

[63]: del table
```

S.02.01.02 2

```
[64]: unique_id = "PRVA_02_2"
pdf_path, page_number, output_pdf_path, output_final_path, codes_path =   
↳extract_paths(master_list=master_list, unique_id=unique_id)
extract_page(input_pdf_path=pdf_path, output_pdf_path=output_pdf_path,  
↳page_number=page_number, password = "")

[65]: table = run_ocr_and_convert_to_df(path=output_pdf_path, api_key=api_key)

[66]: table.columns = ["CODE", "DESCRIPTION", "C0010"]

[67]: set_code_index_and_save(table=table, path=output_final_path)

[68]: del table
```

2.4 Modra zavarovalnica d.d.

S.02.01.02 1

```
[69]: unique_id = "MODRA_02_1"
pdf_path, page_number, output_pdf_path, output_final_path, codes_path =   
↳extract_paths(master_list=master_list, unique_id=unique_id)
extract_page(input_pdf_path=pdf_path, output_pdf_path=output_pdf_path,  
↳page_number=page_number, password = "")

[70]: markdown_text = run_mistral_ocr(output_pdf_path, api_key)

[71]: table = parse_table_to_df(markdown_text, "C0010")

[72]: set_code_index_and_save(table=table, path=output_final_path)

[73]: del table
```

S.02.01.02 2

```
[74]: unique_id = "MODRA_02_2"
pdf_path, page_number, output_pdf_path, output_final_path, codes_path = □
↳extract_paths(master_list=master_list, unique_id=unique_id)
extract_page(input_pdf_path=pdf_path, output_pdf_path=output_pdf_path,□
↳page_number=page_number, password = "")

[75]: table = run_ocr_and_convert_to_df(path=output_pdf_path, api_key=api_key)

[76]: table.columns = ["DESCRIPTION", "CODE", "C0010"]

[77]: set_code_index_and_save(table=table, path=output_final_path)

[78]: del table
```

S.02.01.02 3

```
[79]: unique_id = "MODRA_02_3"
pdf_path, page_number, output_pdf_path, output_final_path, codes_path = □
↳extract_paths(master_list=master_list, unique_id=unique_id)
extract_page(input_pdf_path=pdf_path, output_pdf_path=output_pdf_path,□
↳page_number=page_number, password = "")

[80]: table = run_ocr_and_convert_to_df(path=output_pdf_path, api_key=api_key)

[81]: table.columns = ["DESCRIPTION", "CODE", "C0010"]

[82]: set_code_index_and_save(table=table, path=output_final_path)

[83]: del table
```

2.5 Vzajemna d.d.

S.02.01.02 1

```
[84]: unique_id = "VZAJEMNA_02_1"
pdf_path, page_number, output_pdf_path, output_final_path, codes_path = □
↳extract_paths(master_list=master_list, unique_id=unique_id)
extract_page(input_pdf_path=pdf_path, output_pdf_path=output_pdf_path,□
↳page_number=page_number, password = "")

[85]: table = run_ocr_and_convert_to_df(path=output_pdf_path, api_key=api_key)

[86]: table.columns = ["DESCRIPTION", "CODE", "C0010"]

[87]: set_code_index_and_save(table=table, path=output_final_path)

[88]: del table
```

S.02.01.02 2

```
[89]: unique_id = "VZAJEMNA_02_2"
pdf_path, page_number, output_pdf_path, output_final_path, codes_path =   

↳extract_paths(master_list=master_list, unique_id=unique_id)
extract_page(input_pdf_path=pdf_path, output_pdf_path=output_pdf_path,  

↳page_number=page_number, password = "")

[90]: table = extract_tables_from_pdf(pdf_path=output_pdf_path, page_number=1)

[91]: table = run_ocr_and_convert_to_df(path=output_pdf_path, api_key=api_key)

[92]: table.columns = ["DESCRIPTION", "CODE", "C0010"]

[93]: set_code_index_and_save(table=table, path=output_final_path)

[94]: del table
```

2.6 Grawe zavarovalnica d.d.

S.02.01.02 1

```
[95]: unique_id = "GRAW_02_1"
pdf_path, page_number, output_pdf_path, output_final_path, codes_path =   

↳extract_paths(master_list=master_list, unique_id=unique_id)
extract_page(input_pdf_path=pdf_path, output_pdf_path=output_pdf_path,  

↳page_number=page_number, password = "")

[96]: table = extract_tables_from_pdf(output_pdf_path)

[97]: table.columns = ["DESCRIPTION", "CODE", "C0010"]

[98]: set_code_index_and_save(table=table, path=output_final_path)

[99]: del table
```

S.02.01.02 2

```
[100]: unique_id = "GRAW_02_2"
pdf_path, page_number, output_pdf_path, output_final_path, codes_path =   

↳extract_paths(master_list=master_list, unique_id=unique_id)
extract_page(input_pdf_path=pdf_path, output_pdf_path=output_pdf_path,  

↳page_number=page_number, password = "")

[101]: table = run_ocr_and_convert_to_df(path=output_pdf_path, api_key=api_key)

[102]: table.columns = ["DESCRIPTION", "CODE", "C0010"]

[103]: set_code_index_and_save(table=table, path=output_final_path)

[104]: del table
```

2.7 Zavarovalnica Sava, d.d.

S.02.01.02 1

```
[105]: unique_id = "SAVA_02_1"
pdf_path, page_number, output_pdf_path, output_final_path, codes_path =   
↳extract_paths(master_list=master_list, unique_id=unique_id)
extract_page(input_pdf_path=pdf_path, output_pdf_path=output_pdf_path,  
↳page_number=page_number, password = "")
```

```
[106]: table = run_ocr_and_convert_to_df(path=output_pdf_path, api_key=api_key)
```

```
[107]: table.columns = ["CODE", "DESCRIPTION", "C0010"]
```

```
[108]: set_code_index_and_save(table=table, path=output_final_path)
```

```
[109]: del table
```

S.02.01.02 2

```
[110]: unique_id = "SAVA_02_2"
pdf_path, page_number, output_pdf_path, output_final_path, codes_path =   
↳extract_paths(master_list=master_list, unique_id=unique_id)
extract_page(input_pdf_path=pdf_path, output_pdf_path=output_pdf_path,  
↳page_number=page_number, password = "")
```

```
[111]: table = run_ocr_and_convert_to_df(path=output_pdf_path, api_key=api_key)
```

```
[112]: table.columns = ["CODE", "DESCRIPTION", "C0010"]
```

```
[113]: set_code_index_and_save(table=table, path=output_final_path)
```

```
[114]: del table
```